

**MEDIATEK**

# MT8676 Hypervisor UART User Manual

**Version:** 1.0  
**Release date:** 2024-11-22

Use of this document and any information contained therein is subject to the terms and conditions set forth in Exhibit 1. This document is subject to change without notice.

---

## Version History

| Version | Date       | Author       | Description      |
|---------|------------|--------------|------------------|
| 1.0     | 2024-11-22 | Liliang Chen | Official release |

Table of Contents

Version History ..... 2

Table of Contents ..... 3

List of Figures ..... 3

List of Tables ..... 3

1 Hypervisor UART ..... 4

1.1 Overview ..... 4

1.1.1 Brief Introduction ..... 4

1.1.2 Definitions and Abbreviations ..... 4

1.1 UART Virtualization ..... 5

1.1.1 Virtualization Instructions ..... 5

1.1.2 UART Virtualized Handshake Process ..... 5

1.1.3 UART Virtualization Features ..... 6

1.1.4 Test and Debugging ..... 7

1.3 Passthrough Method ..... 7

1.3.1 Passthrough Instructions ..... 7

1.3.2 Specific Implementation ..... 7

1.3.3 Testing and Debugging ..... 9

1.3.4 Common Problems and Troubleshooting ..... 10

1.3.5 Assistance from MediaTek ..... 10

Exhibit 1 Terms and Conditions ..... 11

List of Figures

Table 1-1. Definitions ..... 4

Table 1-2. Abbreviations ..... 4

Table 1-3. Code path ..... 5

Table 1-4. UART driver code path ..... 7

List of Tables

Figure 1-1. UART virtualization layout diagram ..... 4

Figure 1-2. Handshake flowchart ..... 5

Figure 1-3. SOS modifications ..... 8

Figure 1-4. UOS-DTS modifications ..... 8

Figure 1-5. UOS-KO modifications ..... 9

Figure 1-6. Hypervisor modifications ..... 9

# 1 Hypervisor UART

## 1.1 Overview

### 1.1.1 Brief Introduction

This section describes the software and functions of the MT8676 hypervisor UART, as shown in [Figure 1-1](#).

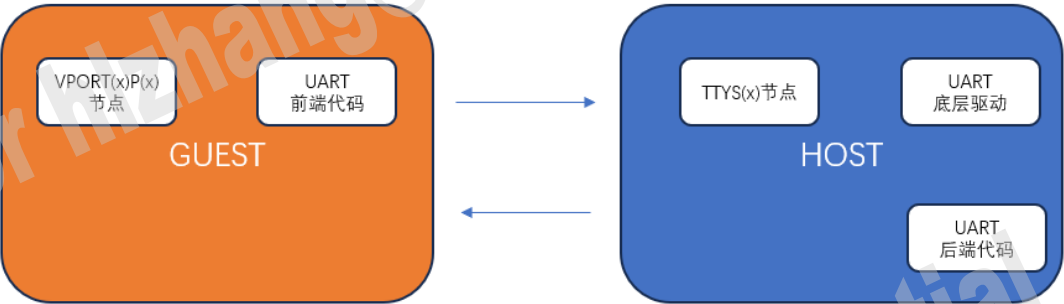


Figure 1-1. UART virtualization layout diagram

### 1.1.2 Definitions and Abbreviations

#### 1.1.2.1 Definitions

Table 1-1. Definitions

| Terms        | Explanation                                      |
|--------------|--------------------------------------------------|
| GUEST        | Front-end systems                                |
| HOST         | Back-end systems                                 |
| TTYS(x)      | Straight-through bound UART nodes                |
| VPORT(x)P(x) | The UART virtual node generated by the front-end |

#### 1.1.2.2 Abbreviations

Table 1-2. Abbreviations

| Abbreviation | Explanation                        |
|--------------|------------------------------------|
| DMA          | Direct Memory access               |
| UART         | Universal Asynchronous Transceiver |

1.1 UART Virtualization

1.1.1 Virtualization Instructions

For multi-OS systems, since multiple external devices need to use UART for communication, but cannot be implemented due to the limitation of the number of UARTs on the platform, the UART module is considered to be virtualized so that different devices can use the UART module in different OS systems, as shown in Table 1-3 for the specific code path.

Table 1-3.Code path

| Front-end Path                       | Back-end Path                                    |
|--------------------------------------|--------------------------------------------------|
| kernel/drivers/char/virtio_console.c | kernel/modules/mt8676/virt/grt/vhost_uart/uart.c |

1.1.2 UART Virtualized Handshake Process

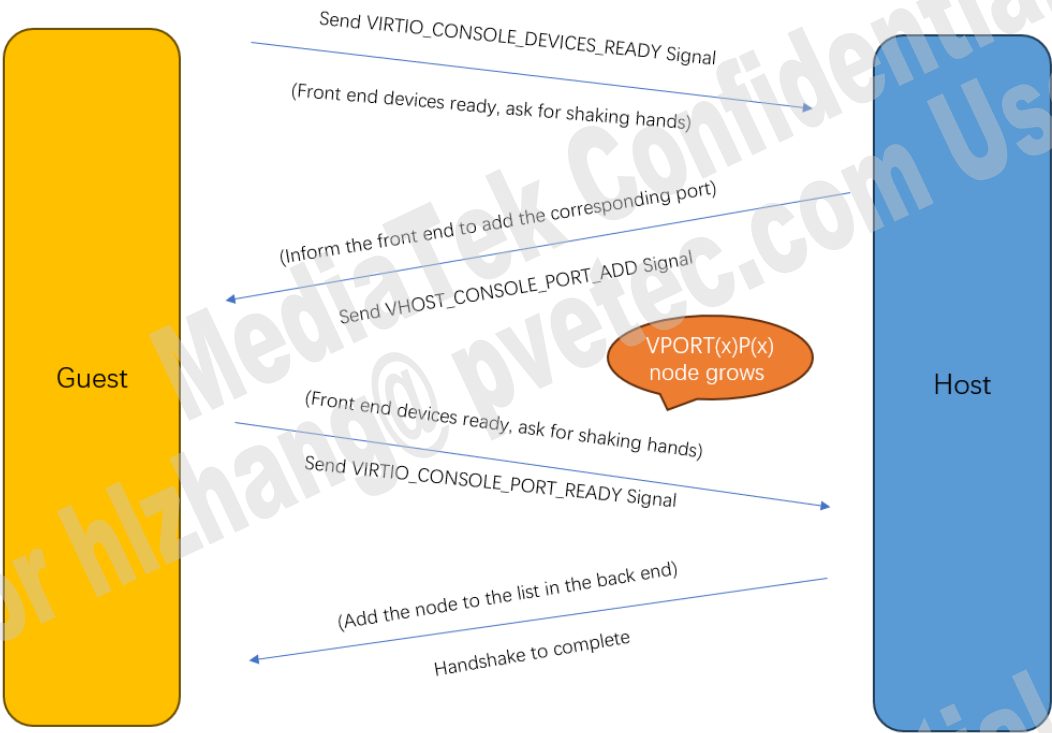


Figure 1-2. Handshake flowchart

For multi-OS system virtualization, the most important thing is to establish a front-end and back-end communication connection. As shown in Figure 1-2, it can be divided into four parts:

1. The front-end (Guest) sends the VIRTIO\_CONSOLE\_DEVICES\_READY event signal to the back-end (Host), indicating that the front-end (Guest) devices are ready.
2. After receiving the signal, the backend (Host) will send a VHOST\_CONSOLE\_PORT\_ADD event signal to the frontend (Guest), indicating that the back-end (Host) has received a signal requesting a connection and needs the front-end to add the corresponding PORT node.

3. When the front-end (Guest) receives the signal, it will send the VIRTIO\_CONSOLE\_PORT\_READY event signal to the back-end (Host), and at this moment, a virtualized node VPORT(x)P(x) will be generated on the front-end (Guest), where x is a randomly generated number.
4. When the host receives the signal, it will add the generated node to the list, which means the handshake process is over.

### 1.1.3 UART Virtualization Features

1. Front-end event signals
  - a) VIRTIO\_CONSOLE\_DEVICE\_READY  
Equipment preparation
  - b) VIRTIO\_CONSOLE\_PORT\_ADD  
Node addition
  - c) VIRTIO\_CONSOLE\_PORT\_REMOVE  
Node removal
  - d) VIRTIO\_CONSOLE\_PORT\_READY  
Node preparation
  - e) VIRTIO\_CONSOLE\_PORT\_OPEN  
Node open
2. Back-end event signals:
  - a) VHOST\_CONSOLE\_DEVICE\_READY  
Equipment preparation
  - b) VHOST\_CONSOLE\_PORT\_ADD  
Node addition
  - c) VHOST\_CONSOLE\_PORT\_REMOVE  
Node removal
  - d) VHOST\_CONSOLE\_PORT\_READY  
Node preparation
  - e) VHOST\_CONSOLE\_PORT\_OPEN  
Node open
  - f) VHOST\_CONSOLE\_PORT\_READ  
Node read

### 1.1.4 Test and Debugging

1. Check whether a UART virtual node is generated on the front-end.

```
find . -name vport*
```

2. To get the UART data, you can use the following command to receive the UART data and print it out in the console.

```
cat /dev/vport*
```

3. The UART can send data; you can use the following command to have the UART send the string "123".

```
echo 123 > /dev/vport*
```

## 1.3 Passthrough Method

### 1.3.1 Passthrough Instructions

For multiple OSs, there is a set of MTK UART driver codes for each OS, as shown in [Table 1-4](#).

The virt mode of UART currently supports read/write/on/off operations, and you need to modify the baud rate, enable soft streaming, and enable hard streaming through Passthrough mode, that is, enable the UART driver code under the corresponding OS.

**Table 1-4. UART driver code path**

| System  | UART Path                                                                        | APDMA Path                                                                          |
|---------|----------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| Android | kernel/kernel_device_modules-6.1/drivers/tty/serial/8250/8250_mtk.c              | Kernel/kernel_device_modules-6.1/drivers/dma/mediatek/mtk-uart-apdma.c              |
| Yocto   | src/kernel/linux/v6.1_mt8676/co_device_module/drivers/tty/serial/8250/8250_mtk.c | src/kernel/linux/v6.1_mt8676/co_device_module/drivers/dma/mediatek/mtk-uart-apdma.c |

### 1.3.2 Specific Implementation

Take the LLA system UART2 of SOS (Yocto) passthrough to UOS (Android) as an example.

#### 1.3.2.1 SOS (Yocto) Modifications

The SOS (Yocto) part is slightly changed, just make sure that the UART2 node property status in the auto8676p1\_64\_hyp\_sos.dts file is changed from "okay" to "disabled". As shown in [Figure 1-3](#).

```
diff --git a/recipes-kernel/linux/files/auto8676p1_64_hyp_sos.dts b/recipes-kernel/linux/files/auto8676p1_64_hyp_sos.dts
index c2df6b5..75ffc9d 100644
--- a/recipes-kernel/linux/files/auto8676p1_64_hyp_sos.dts
+++ b/recipes-kernel/linux/files/auto8676p1_64_hyp_sos.dts
@@ -393,7 +393,7 @@
&uart2 {
    pinctrl-names = "default";
    pinctrl-0 = <&uart2_pins_default>;
-    status = "okay";
+    status = "disabled";
};
```

Figure 1-3. SOS modifications

### 1.3.2.2 UOS (Android) Modifications

There are two modifications for UOS (Android):

#### 1. dts revise

The UOS (Android) system needs to modify the auto8676p1\_64\_bsp\_vm.dts file, enable the UART2 node, and switch the pin corresponding to UART2 to UART mode, so that the pin will automatically switch to UART mode after the platform is turned on. Figure 1-4 shows the modifications.

```
diff --git a/arch/arm64/boot/dts/mediatek/auto8676p1_64_bsp_vm.dts b/arch/arm64/boot/dts/mediatek/auto8676p1_64_bsp_vm.dts
index db4023b90c89..9325e59e8b8f 100644
--- a/arch/arm64/boot/dts/mediatek/auto8676p1_64_bsp_vm.dts
+++ b/arch/arm64/boot/dts/mediatek/auto8676p1_64_bsp_vm.dts
@@ -1632,6 +1632,27 @@
    status = "disabled";
};

+&pio {
+    uart2_pins_default: uart2-pins-default {
+        pins-rx {
+            pinmux = <PINMUX_GPIO117_FUNC_URXD2>;
+            input-enable;
+            bias-pull-up;
+        };
+        pins-tx {
+            pinmux = <PINMUX_GPIO121_FUNC_UTXD2>;
+        };
+        pins-rts {
+            pinmux = <PINMUX_GPIO162_FUNC_URTS2>;
+            output-enable;
+        };
+        pins-cts {
+            pinmux = <PINMUX_GPIO161_FUNC_UCTS2>;
+            input-enable;
+        };
+    };
+};
+
&uart0 {
    status = "disabled";
};
@@@ -1641,7 +1662,9 @@@ &uart1 {
};

&uart2 {
-    status = "disabled";
+    pinctrl-names = "default";
+    pinctrl-0 = <&uart2_pins_default>;
+    status = "okay";
};
```

Figure 1-4. UOS-DTS modifications

#### 2. ko\_table revise

Since the UART-related ko files have not been added on the UOS (Android) side, it is also necessary to add them to the ko\_order\_table file, as shown in Figure 1-5.

```
diff --git a/ko_order_table.csv b/ko_order_table.csv
index 7c9b7ee..e0b0dcb 100644
--- a/ko_order_table.csv
+++ b/ko_order_table.csv
@@ -127,6 +127,9 @@ mtk-pmic-keys.ko,./kernel_device_modules-6.1/drivers/input/keyboard/mtk-pmic-k
mtk-kpd.ko,./kernel_device_modules-6.1/drivers/input/keyboard/mtk-kpd.ko,ramdisk,Y,Y,user/userdebug/eng
i2c-mt65xx.ko,./kernel_device_modules-6.1/drivers/i2c/busses/i2c-mt65xx.ko,ramdisk,Y,Y,user/userdebug/eng
spi-mt65xx.ko,./kernel_device_modules-6.1/drivers/spi/spi-mt65xx.ko,ramdisk,Y,Y,user/userdebug/eng
+mtk-uart-apdma.ko,./kernel_device_modules-6.1/drivers/dma/mediatek/mtk-uart-apdma.ko,ramdisk,Y,Y,user/userdebug/eng
+uarthub_drv.ko,./kernel_device_modules-6.1/drivers/misc/mediatek/uarthub/uarthub_drv.ko,vendor,Y,Y,user/userdebug/eng
+8250_mtk.ko,./kernel_device_modules-6.1/drivers/tty/serial/8250/8250_mtk.ko,ramdisk,Y,Y,user/userdebug/eng
+extdev_io_class.ko,./kernel_device_modules-6.1/drivers/misc/mediatek/subpmic/extdev_io_class.ko,ramdisk,Y,Y,user/userdebug/eng
subpmic-dbg.ko,./kernel_device_modules-6.1/drivers/misc/mediatek/subpmic/subpmic-dbg.ko,ramdisk,Y,Y,user/userdebug/eng
rt5133-regulator.ko,./kernel_device_modules-6.1/drivers/regulator/rt5133-regulator.ko,ramdisk,Y,Y,user/userdebug/eng
```

Figure 1-5. UOS-KO modifications

### 1.3.2.3 Hypervisor Modifications

Since the hypervisor enables all interrupt numbers on the SOS (Yocto) side, you need to enable the interrupt number used by the UOS (Android) side, so you need to enable the interrupt number of UART2 on the UOS side in the uos\_alps\_pv8676.lua file, as shown in Figure 1-6.

```
if Android_USB_SWITCH_value == "1" then
    print("use android usb")
    interrupt = {48,70,192,193, 231, 255, 256, 284, 634, 635, 645, 646, 921, 923, 924, 929}
    for i = 1, #interrupt do
        uos_config:setVgicIrq(interrupt[i])
    end
end
```

Figure 1-6. Hypervisor modifications

1. The interrupt number of UART2 can be queried as 252 in the MT6897.dts file, and 32 needs to be added to the LUA file, so the interrupt number of UART2 is 284 in the uos\_alps\_pv8676.lua file.
2. When UART2 enables DMA mode, DMA has two channels exist: DMA TX and DMA RX, so the DMA lookup table of UART2 can be queried as 223, 224, and by adding 32, it can be obtained as 255,256 in the lua file.

### 1.3.3 Testing and Debugging

Perform the following operations under the OS with the UART driver enabled for testing.

1. Set baud rate to 921600, 8 bits of data, 1 bit of stop, no parity, e.g., UART2

```
stty -F /dev/ttyS2 ispeed 921600 ospeed 921600 cs8
```

2. Obtain the current UART baud rate and data bit configuration information

```
stty -F /dev/ttyS2 -a
```

3. UART obtains data; you can use the following command to receive UART2 data and print it out in the console.

```
cat /dev/ttyS2
```

4. UART to send data, you can use the following command to have UART2 send the string "123"

```
echo 123 > /dev/ttyS2
```

### 1.3.4 Common Problems and Troubleshooting

#### 1.3.4.1 Android Cannot Boot after the Modifications.

1. Please check whether the uos\_alps\_pv8676.lua file in the platform (non-codebase) contains the UART interrupt number, which can be typed in the shell:

```
vi /vendor/etc/hyper_android/uos_alps_xx.lua
```

It is important to ensure that the lua file in the platform contains the interrupt number required by UART.

2. If a log similar to the following appears in the boot log:

```
Init: [name:main&]8250_mtk: Unknown symbol UARTHUB_md_adsp_fifo_ctrl (err -2)
```

Verify that the ko\_order\_table file has been modified correctly and confirm that the three KOs have been added as illustrated in [Figure 1-5](#).

#### 1.3.4.2 UART Test Failed after the Modifications

1. Please check whether the modified dts for UOS/SOS is correct. The previous text uses the L+A+L system as an example. If it is an L+A system, the dts file will be different. Please confirm whether the dts file has been correctly modified.
2. Please check whether the dts configuration is correct. Use Linux commands to check whether the UART pin mode has been switched to UART mode. To view the status of the PIN in the kernel, you can enter the following command to query:

```
# cat /proc/mtk_gpio/soc.pinctrl
# cd /proc/mtk_gpio/
# cat soc.pinctrl
```

e.g.,

```
# cat /proc/mtk_gpio/soc.pinctrl
PIN: [MODE] [DIR] [DOUT] [DIN] [PULL_EN] [PULL_SEL] [IES] [SMT] [DRIVE] ( [R1] [R0] )
0: 0 0 0 0 1 0 1 0 0
1: 0 0 0 0 1 0 1 0 0
2: 0 1 1 1 1 0 1 0 0
3: 6 0 0 0 1 0 1 0 0
```

### 1.3.5 Assistance from MediaTek

If the issue cannot be resolved after the aforementioned troubleshooting steps, you can ask help from MediaTek with provide relevant logs, DTS files, and the output of cat mt\_gpio provided.

## Exhibit 1 Terms and Conditions

Your access to and use of this document and the information contained herein (collectively this “Document”) is subject to your (including the corporation or other legal entity you represent, collectively “You”) acceptance of the terms and conditions set forth below (“T&C”). By using, accessing or downloading this Document, You are accepting the T&C and agree to be bound by the T&C. If You don’t agree to the T&C, You may not use this Document and shall immediately destroy any copy thereof.

This Document contains information that is confidential and proprietary to MediaTek Inc. and/or its affiliates (collectively “MediaTek”) or its licensors and is provided solely for Your internal use with MediaTek’s chipset(s) described in this Document and shall not be used for any other purposes (including but not limited to identifying or providing evidence to support any potential patent infringement claim against MediaTek or any of MediaTek’s suppliers and/or direct or indirect customers). Unauthorized use or disclosure of the information contained herein is prohibited. You agree to indemnify MediaTek for any loss or damages suffered by MediaTek for Your unauthorized use or disclosure of this Document, in whole or in part.

MediaTek and its licensors retain titles and all ownership rights in and to this Document and no license (express or implied, by estoppels or otherwise) to any intellectual propriety rights is granted hereunder. This Document is subject to change without further notification. MediaTek does not assume any responsibility arising out of or in connection with any use of, or reliance on, this Document, and specifically disclaims any and all liability, including, without limitation, consequential or incidental damages.

THIS DOCUMENT AND ANY OTHER MATERIALS OR TECHNICAL SUPPORT PROVIDED BY MEDIATEK IN CONNECTION WITH THIS DOCUMENT, IF ANY, ARE PROVIDED “AS IS” WITHOUT WARRANTY OF ANY KIND, WHETHER EXPRESS, IMPLIED, STATUTORY, OR OTHERWISE. MEDIATEK SPECIFICALLY DISCLAIMS ALL WARRANTIES OF MERCHANTABILITY, NON-INFRINGEMENT, FITNESS FOR A PARTICULAR PURPOSE, COMPLETENESS OR ACCURACY AND ALL WARRANTIES ARISING OUT OF TRADE USAGE OR OUT OF A COURSE OF DEALING OR COURSE OF PERFORMANCE. MEDIATEK SHALL NOT BE RESPONSIBLE FOR ANY MEDIATEK DELIVERABLES MADE TO MEET YOUR SPECIFICATIONS OR TO CONFORM TO A PARTICULAR STANDARD OR OPEN FORUM.

Without limiting the generality of the foregoing, MediaTek makes no warranty, representation or guarantee regarding the suitability of its products for any particular purpose, nor does MediaTek assume any liability arising out of the application or use of any product, circuit or software. You agree that You are solely responsible for the designing, validating and testing Your product incorporating MediaTek’s product and ensure such product meets applicable standards and any safety, security or other requirements.

The above T&C and all acts in connection with the T&C or this Document shall be governed, construed and interpreted in accordance with the laws of Taiwan, without giving effect to the principles of conflicts of law.